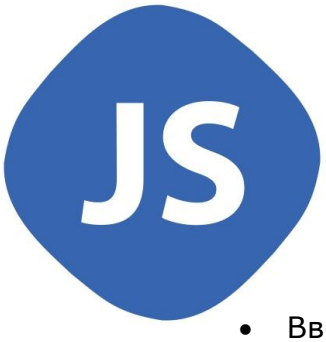


Файлы Cookie

- Введение в Cookie
- Область видимости и время жизни Cookie
- Удаление Cookie
- Cookie и JSON
- Сторонние Cookie и безопасность
- Дивертисмент на тему аутентификации
- Функции для работы с Cookie

Введение в Cookie

Cookie (куки) - это данные, хранящиеся в небольших текстовых файлах, на **компьютере пользователя**. Cookie являются частью HTTP-протокола.

После отправки клиенту (в браузер) запрошенной страницы веб-сервер завершает подключение, и все данные о выполнившем запрос пользователе теряются. Cookie были изобретены как решение этой проблемы.

Алгоритм работы достаточно прост:

- когда пользователь посещает веб-страницу, данные о нем могут запоминаться в файле cookie;
- в следующий раз, когда пользователь посещает страницу, сервер увидит данные файла cookie и *вспомнит* о пользователе.

Cookie обычно устанавливаются веб-сервером при помощи заголовка **Set-Cookie**. Браузер будет автоматически добавлять **установленный cookie** в каждый запрос на тот же домен при помощи заголовка **Cookie**.

> ***Важно.*** Когда пользователь запрашивает веб-страницу с сервера, файлы cookie, **принадлежащие этой странице**, добавляются в запрос.



Таким образом, сервер получает необходимые данные о запросившем ресурс пользователе.

Файлы **cookie** имеют директивы:

- **name** - имя;
- **value** - значение;
- **secure** – безопасность;
- **expires** – время жизни;
- **path** - путь;
- **domain** – домен;
- **options** - объект с дополнительными опциями.

Cookie имеют множество особенностей и тонкостей в использовании.

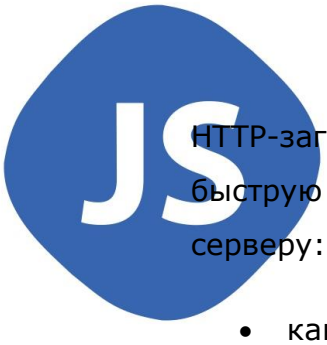
> ***Важно***. Алгоритм работы с файлами Cookie в **клиентских** и **серверных** сценариях абсолютно **различен**.

Серверный сценарий отправляет **запрос** на создание файла cookie браузером клиента. Браузер создает файл cookie и отправляет данные созданного файла в **ответе** серверу. Этот **обмен данными** (запрос-ответ) происходит в **HTTP-заголовках** браузера.

> ***Важно***. Именно поэтому серверный сценарий не имеет доступ к **cookie** установленным в **текущем скрипте**. Сервер не устанавливает, он просит установить, и получит реакцию на просьбу только после получения ответа от браузера (т.е., при перезагрузке страницы или новом запросе).

HTTP-заголовки (заголовки браузера) служат оболочкой для передачи информации между браузером и сервером сайта.

Когда браузер запрашивает веб-страницу, он отправляет HTTP-запрос на сервер, запрашивающий эту страницу. Это сообщение и ответ сервера содержат HTTP-заголовки.



HTTP-заголовки обеспечивают бесперебойную, безопасную и максимально быструю связь между браузером и сайтом. Например, они могут сообщить серверу:

- какой тип веб-страницы или файла может обрабатывать браузер;
- как должно отображаться содержимое;
- как защитить связь между браузером и сервером;
- какие данные cookie использует удаленный клиент;
- и др...

Формат заголовков HTTP представляет собой простую структуру ключ-значение, предназначенную для удобства разбора и интерпретации как клиентами, так и серверами.

Перед выполнением разбора кодов демонстрационных примеров важно помнить следующее.

Первое. Ни один из примеров не будет работать, если в вашем браузере **отключена** поддержка локальных файлов cookie.

> ***Второе***. Выполняя демонстрационные примеры не забывайте **регулярно очищать кеш браузера** от неактуальных файлов **cookie**. В противном случае рискуете получить устаревшие данные и неверное представление о поведении файлов cookie и работе сценария.

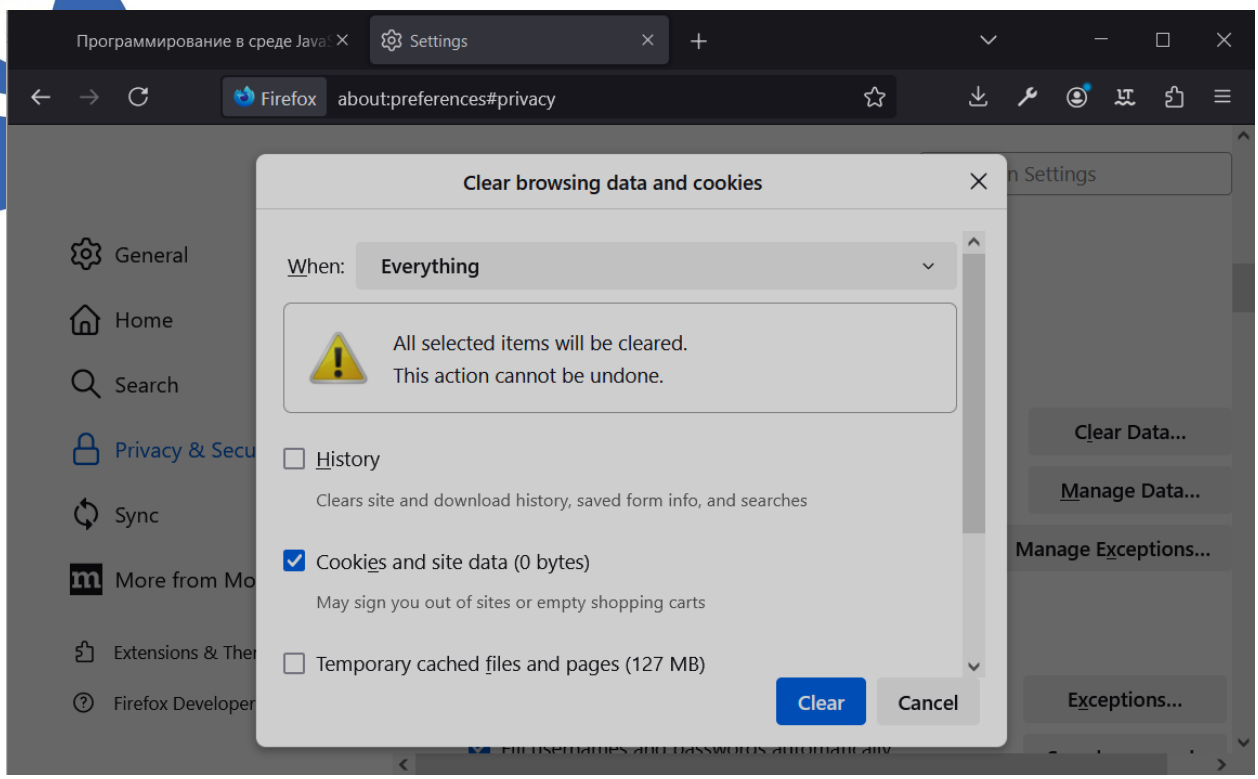


Рис.1. Регулярно удаляйте неактуальные данные cookie

Запись в document.cookie

Для сохранения данных необходимо выполнить запись в **document.cookie**. Но это не просто свойство данных, а **аксессор** (геттер/сеттер). Присваивание к нему обрабатывается особым образом.

Пример установки cookie с именем **login** и значением DenisDream:

```
// установка cookie

document.cookie = "login=DenisDream";
```

Запись в **document.cookie** обновит только упомянутые в ней cookie, но при этом **не затронет все остальные**.

Чтение из document.cookie

Получить доступ к cookie можно также используя свойство **document.cookie**.

```
// чтение cookie
```

```
document.write(document.cookie); // вывод cookie в браузер
console.log(document.cookie); // вывод cookie в консоль
alert(document.cookie); // вывод cookie в диалоговое окно
```

Значение **document.cookie** состоит из пар **ключ=значение**, разделённых символом **;**. Каждая пара представляет собой отдельный **cookie**.

Чаще всего разработчик имеет дело со **строкой cookie**, хранящейся в формате приведенном на рисунке 2.

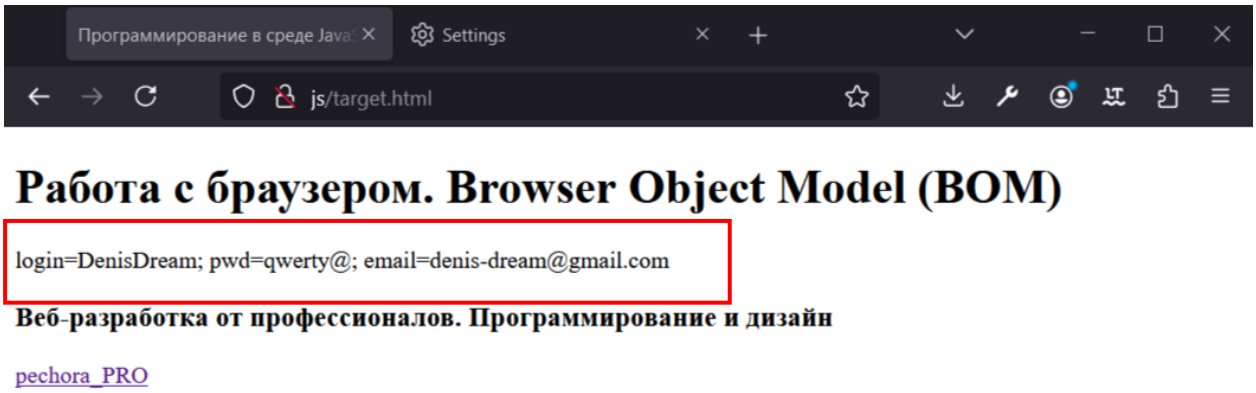


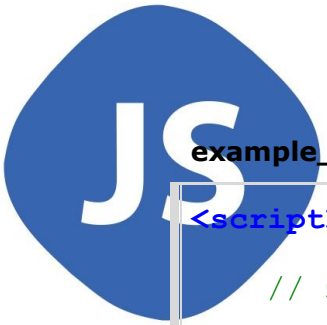
Рис.2. Формат представления данных cookie

Создадим файл cookie.

example_1.

```
<script>
    // проверим начальное содержимое cookie
    console.log(document.cookie); // <empty string>
    // присвоим cookie новое значение
    document.cookie = "login=DenisDream";
    // проверим содержимое cookie
    console.log(document.cookie, '<p>'); // login=DenisDream
</script>
```

В демонстрационном примере **example_2** создадим несколько файлов cookie.



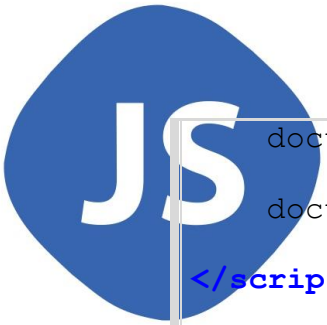
example_2.

```
<script>
    // запись в cookie
    document.cookie = "login=DenisDream";
    // проверка содержимого cookie
    document.write(document.cookie, '<p>'); // login=DenisDream
    // еще одна запись в cookie
    document.cookie = "pwd=qwerty@";
    // проверка содержимого cookie
    // старое значение никуда не делось
    document.write(document.cookie, '<p>'); // login=DenisDream;
    pwd=qwerty@
    // и еще одна запись в cookie
    document.cookie = "email=denis-dream@gmail.com";
    document.write(document.cookie, '<p>'); // login=DenisDream;
    pwd=qwerty@; email=denis-dream@gmail.com
</script>
```

Файлы cookie нужны для передачи данных между страницами сайта. В следующем примере используем переход между страницами с передачей данных cookie.

example_3. **example_3.html**

```
<!-- ссылка для проверки содержимого cookie -->
<a href="target.html">Go-go</a>
<script>
    // сохраним в cookie три параметра
    document.cookie = "login=DenisDream";
```



```
    document.cookie = "pwd=qwerty@";

    document.cookie = "email=denis-dream@gmail.com";

</script>
```

example_3. target.html

```
<script>

    // вывод данных cookie

    document.write(document.cookie);

    console.log(document.cookie);

</script>
```

Данные **cookie** можно собирать от файла к файлу.

example_4. example_4-1.html

```
<a href="example_4-2.html">Go example_4-2.html</a><p>

<script>

    // запишем в cookie ключ login и его значение

    document.cookie = "login=DenisDream";

</script>
```

example_4. example_4-2.html

```
<a href="example_4-3.html">Go example_4-3.html</a><p>

<script>

    // выведем данные cookie сайта в виде строки

    document.write(document.cookie);

    // запишем в cookie параметр pwd

    document.cookie = "pwd=qwerty@";
```



```
</script>
```

example_4. example_4-3.html

```
<a href="target.html">Go target.html</a><p>
<script>
    // выведем данные cookie сайта в виде строки
    document.write(document.cookie);
    // запишем в cookie параметр email
    document.cookie = "email=denis-dream@gmail.com";
</script>
```

Чтобы получить доступ к определённому **cookie**, достаточно разбить строку из **document.cookie** по символу **;**, и затем найти нужный ключ. Для этого мы можем использовать как регулярные выражения, так и функции обработки массивов.

example_4. target.html

```
<script>
    // выведем данные cookie сайта в виде строки
    document.write(document.cookie, '<p>');
    // разобъем строку по парам параметр=значение
    let cookies = document.cookie.split('; ');
    document.write('<h3>Вывод содержимого document.cookie в
    цикле</h3>')
    // выведем данные cookie в цикле
    for (let cookie of cookies) {
        // сохраним пару параметр=значение в соответствующие
        переменные
```

```
        let [key, val] = cookie.split('=');

        document.write(key, ' -> ', val, '<p>');
    }
</script>
```

Значением ключа **cookie** может быть строка, содержащая **символы кириллицы** или **специальный символ-разделитель** параметров – **;**. Тогда строка должна быть закодирована. Собственно, строка должна быть закодирована во всех случаях хранения потенциально опасных символов.

example_5. **example_5-1.html**.

```
<script>
    // значение переменной citate содержит символ: ;
    let citate = '"Не смотри на часы; делай то, что они делают. Продолжай идти". – Сэм Левенсон';
    // запись в cookie
    document.cookie = `citate=${citate}`;
    // анализируем вывод
    document.write('<h2>', document.cookie, '</h2>');
</script>
```

Пример использования cookie с кодированием значений.

example_5. **example_5-2.html**

```
<script>
    // значение переменной citate содержит символ: ;
    let citate = '"Не смотри на часы; делай то, что они делают. Продолжай идти". – Сэм Левенсон';
    // кодируем строку перед записью в cookie
    citate = encodeURIComponent(citate);
```

```
    document.cookie = `citate=${citate}`;
    // анализируем вывод
    document.write('<h2>', decodeURIComponent(document.cookie),
    '</h2>');
</script>
```

Символ-разделитель понятно. Но почему необходимо кодировать символы кириллицы.

Необходимо помнить

Все пересылаемые по протоколу HTTP данные будут **кодированы**. Если вы не сделаете это самостоятельно, браузер сделает это за вас. Точнее так, каждый браузер будет самостоятельно решать – каким образом выполнить кодирование / декодирование пересылаемых данных.

Исходя из этого, правильное решение – предупреждающее использование функций **encodeURI** / **encodeURIComponent**.

example_6. **example_6.html**

```
<a href="set-cookie.html">Установить Cookie</a><p>
<script>
    // вывод cookie
    // document.write('<h2>',
    decodeURIComponent(document.cookie), '</h2>');
    document.write('<h2>', document.cookie, '</h2>');
</script>
```

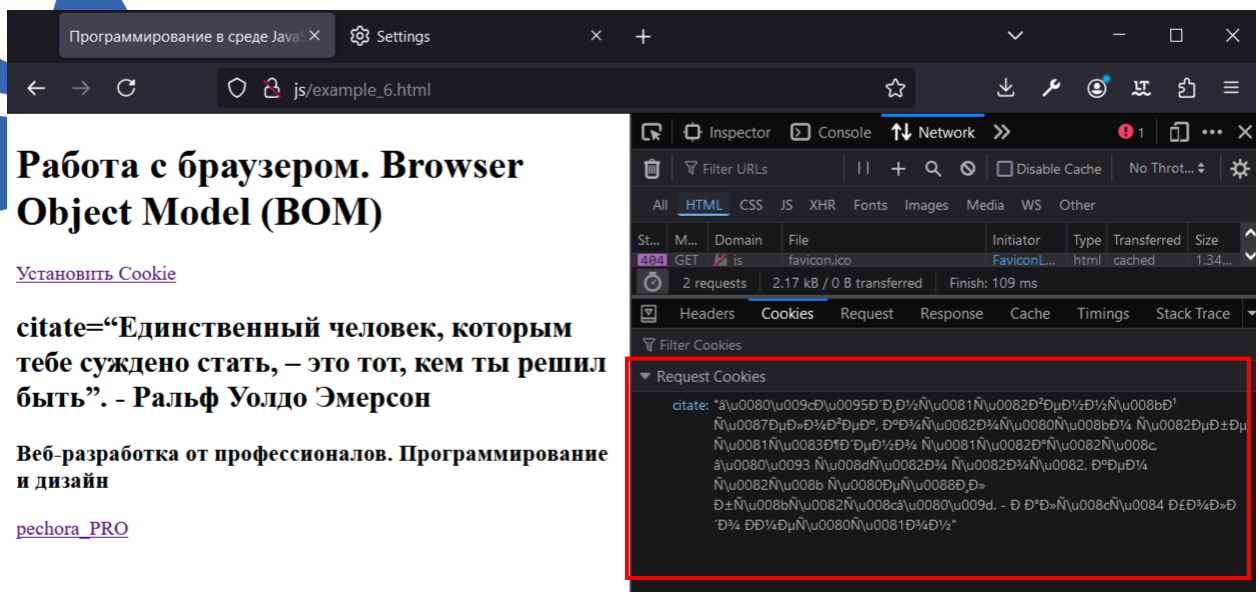


Рис.3. Кодировка пересылаемых данных

example_6. set-cookie.html

```
<script>

    // сохраняем cookie

    let citate = '"Единственный человек, которым тебе суждено
    стать, – это тот, кем ты решил быть". - Ральф Уолдо Эмерсон';

    // citate = encodeURIComponent(citate);

    document.cookie = `citate=${citate}; path=/`;

    // переадресуем на основную страницу

    window.location.assign(window.location.origin +
    '/example_6.html');

</script>
```



Область видимости и время жизни Cookie

Рассмотрим наиболее используемые директивы.

Директива path

URL-префикс пути, cookie будут доступны для страниц этой директории. Должен быть абсолютным. По умолчанию используется текущий путь.

Синтаксис директивы:

path=/mypath

Если куки установлено с **path=/admin**, то оно будет доступно на страницах:

- /admin и
- /admin/person,

но не на страницах

- /person или
- /admin-person.

Как правило, указывают в качестве пути корень **path=/**, чтобы наше cookie было доступно **на всех страницах сайта**.

В следующем примере обратие внимание на определение области видимости (директории) файлов cookie.

example_7. **example_7.html**

```
<h3>Файлы Cookie установлены, перейдите по ссылкам для проверки document.cookie</h3>
<a href="target.html">Go target.html</a><p>
<a href="cookie/target.html">Go cookie/target.html</a><p>
```

```
<script>
    // cookie установлены для директории /cookie
    document.cookie = "login=DenisDream; path=/cookie";
    document.cookie = "pwd=qwerty@; path=/cookie";
    document.cookie = "email=denis-dream@gmail.com;
    path=/cookie";
</script>
```

Просмотр файлов cookie **текущей директории**.

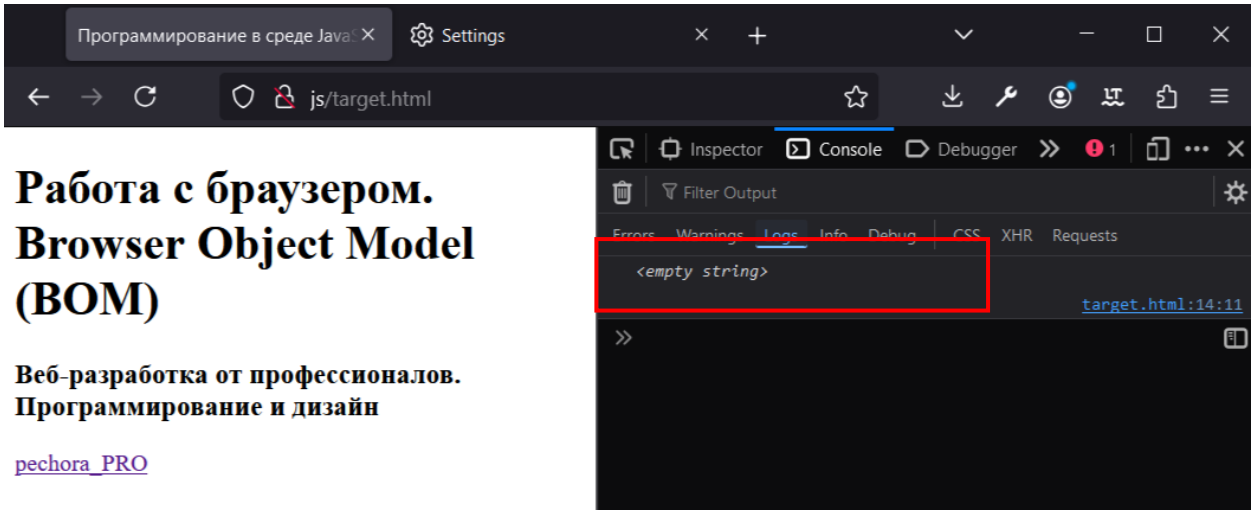


Рис.4. Cookie – пустая строка

example_7. target.html

```
<script>
    // просмотр cookie сайта текущей директории
    console.log(document.cookie);
</script>
```

Просмотр файлов cookie директории **cookie**, **path=/cookie**.

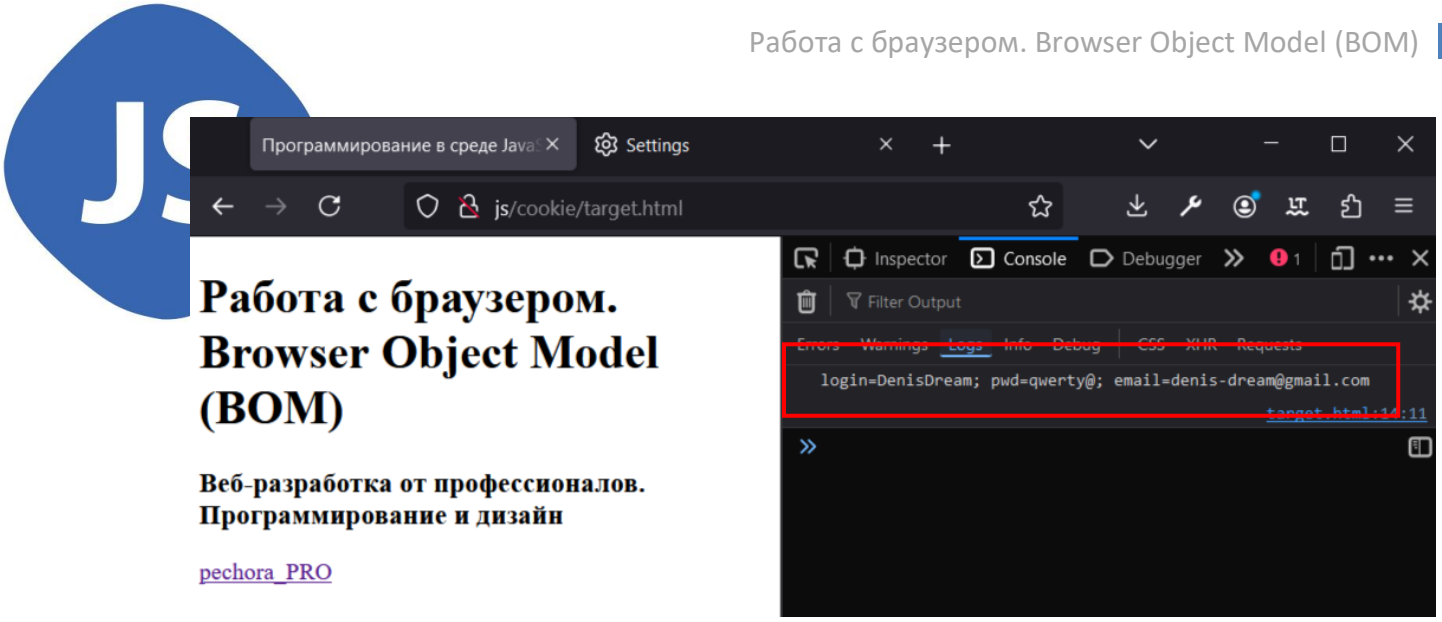


Рис.5. Данные cookie для директории cookie

example_7. cookie/target.html

```
<script>

    // просмотр cookie сайта директории /cookie

    console.log(document.cookie);

</script>
```

Усложним пример.

example_8. example_8-1.html

```
<!-- просмотр cookie директории admin -->

<a href="admin/target.html">Go admin page</a><p>

<!-- просмотр cookie директории user -->

<a href="user/target.html">Go user page</a><p>

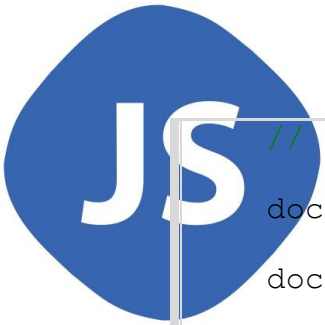
<script>

    // cookie для директории user

    document.cookie = `login=Fluffy; path=/user`;

    document.cookie = `pwd=12345; path=/user`;

    document.cookie = `role=user; path=/user`;
```



```
    // cookie для директории admin
    document.cookie = `login=DenisDream; path=/admin`;
    document.cookie = `pwd=qwerty@; path=/admin`;
    document.cookie = `role=admin; path=/admin`;
    // общий cookie
    document.cookie = `msg=Hello; path=/`;
</script>
```

example_8. admin/target.html

```
<script>
    // просмотр cookie сайта в директории /admin
    document.write(document.cookie);
</script>
```

example_8. user/target.html

```
<script>
    // просмотр cookie сайта директории /user
    document.write(document.cookie);
</script>
```

В примере **example_8-2** обратите внимание на двойное определение ключа **role**.

example_8. example_8-2.html

```
<!-- просмотр cookie директории admin -->
<a href="admin/target.html">Go admin page</a><p>
<!-- просмотр cookie директории user -->
```

```
<a href="user/target.html">Go user page</a><p>
<script>
    // cookie для директории user
    document.cookie = `login=Fluffy; path=/user`;
    document.cookie = `pwd=12345; path=/user`;
    document.cookie = `role=user; path=/user`;
    // cookie для директории admin
    document.cookie = `login=DenisDream; path=/admin`;
    document.cookie = `pwd=qwerty@; path=/admin`;
    document.cookie = `role=admin; path=/admin`;
    // обратите внимание на еще одно определение role
    document.cookie = `role=moderator; path=/`;
</script>
```

Директивы expires, max-age

По умолчанию, если cookie не имеют ни одного из этих параметров, то они удалятся при закрытии браузера. Такие куки называются **сессионными** (session cookies).

Для продления времени жизни cookie можно установить значение опций **expires** или **max-age**.

Синтаксис директивы:

```
expires=Tue, 19 Jan 2038 03:14:07 GMT
```

Дата истечения срока действия cookie, после чего браузер удалит его автоматически.

Дата должна быть точно в этом формате, во временной зоне GMT. Чтобы получить правильную дату можно использовать **date.toUTCString**.



Например, мы можем установить срок действия cookie на один день.

```
// один день от текущей даты
let date = new Date(Date.now() + 86400 * 1000);
date = date.toUTCString();
document.cookie = "login=DenisDream; expires=" + date;
```

Если мы установим в **expires** прошедшую дату, то куки будет удалено.

example_9. **example_9.html**

```
<a href="target.html">Go-go</a><p>
<script>
    /*
        сутки = 24 * 60 * 60 * 1000 || 86400 * 1000 (мсек.)
    */
    // создаем объект Date + 1 день
    let date = new Date(Date.now() + 86400 * 1000);
    console.log(date); // Date
    date = date.toUTCString();
    console.log(date);  // string
    // сохраняем cookie с временем жизни - 1 день
    document.cookie = `login=DenisDream; path=/;
    expires=${date}`;
    document.cookie = `pwd=qwerty@; path=/; expires=${date}`;
    document.cookie = `email=denis-dream@gmail.com; path=/;
    expires=${date}`;
</script>
```

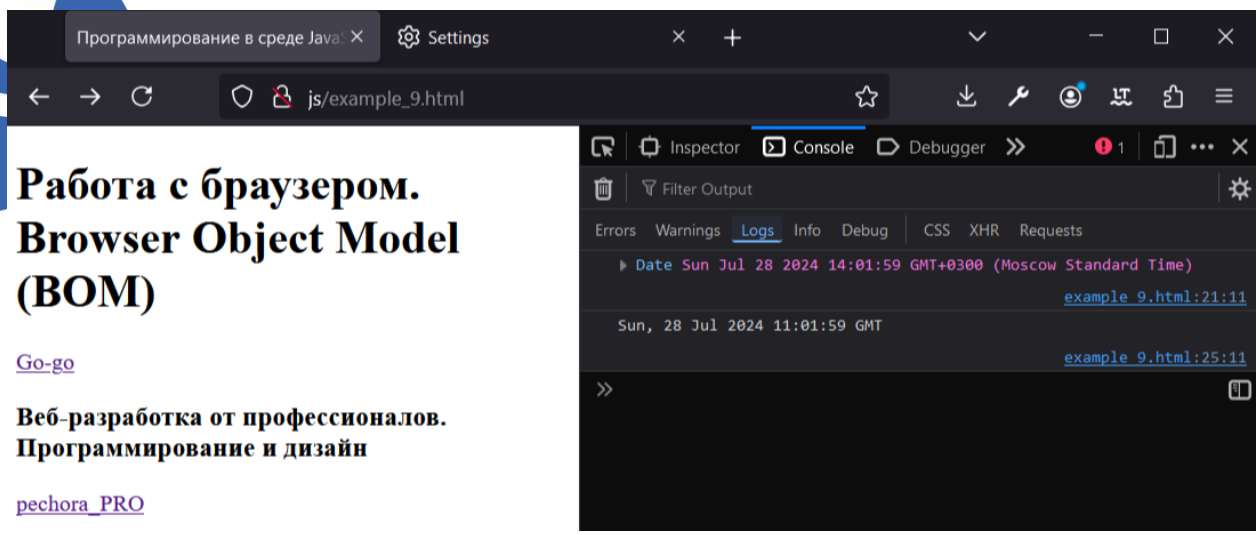


Рис.6. Определение времени жизни cookie

example_9. target.html

```
<script>

    // просмотр cookie

    console.log(document.cookie);

</script>
```

В следующем демонстрационном примере установим несколько cookie с разным временем жизни.

example_10. example_10.html

```
<a href="target.html">Go-go</a><p>

<script>

    // создадим временные промежутки жизни файлов cookie

    let date1 = new Date(Date.now() + 5 * 1000); // 5 сек.

    let date2 = new Date(Date.now() + 10 * 1000); // 10 сек.

    let date3 = new Date(Date.now() + 15 * 1000); // 15 сек.

    // представим время к строке

    date1 = date1.toUTCString();
```

```
    date2 = date2.toUTCString();
    date3 = date3.toUTCString();
    // сохраняем cookie с ограниченным временем жизни
    document.cookie = `login=DenisDream; path=/;
    expires=${date1}`;
    document.cookie = `pwd=qwerty@; path=/; expires=${date2}`;
    document.cookie = `email=denis-dream@gmail.com; path=/;
    expires=${date3}`;
</script>
```

Обновите несколько раз целевую страницу, убедитесь, что после истечения времени жизни cookie были удалены.

example_10. **target.html**

```
<script>
    // обновим несколько раз страницу, проверим состояние cookie
    сайта
    document.write(document.cookie);
</script>
```

Директива **max-age** - альтернатива **expires**, определяет срок действия cookie в **секундах** с **текущего момента** (что очень часто гораздо удобнее).

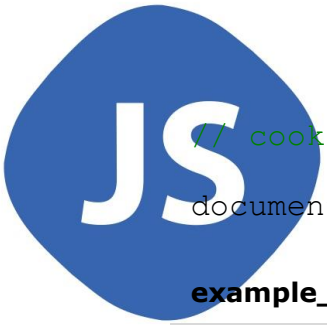
Синтаксис директивы:

```
max-age=3600
```

Обратите внимание, значение директивы указывается в секундах.

Если задан ноль или отрицательное значение, то куки будет удалено:

```
// cookie будет удален через один час
document.cookie = "login=DenisDream; max-age=3600";
```



```
// cookie будет удален немедленно
document.cookie = "login=DenisDream; max-age=0";
```

example_11. **example_11.html**

```
<a href="target.html">Go-go</a><p>
<script>
    // определяем время в секундах
    let date1 =  5; // 5 сек.
    let date2 = 10; // 10 сек.
    let date3 = 15; // 15 сек.
    // устанавливаем cookie с ограниченым времменем жизни
    document.cookie = `login=DenisDream; path=/; max-
    age=${date1}`;
    document.cookie = `pwd=qwerty@; path=/; max-age=${date2}`;
    document.cookie = `email=denis-dream@gmail.com; path=/; max-
    age=${date3}`;
</script>
```

Для автоматического отслеживанния состояния **document.cookie** используем таймер **setInterval**().

example_11. **target.html**

```
<div id="console"></div>
<script>
    // с помощью функции setInterval каждые 2 сек. выводим файлы
    cookie
    let int = setInterval( () => {
        if(document.cookie == ''){
            clearInterval(int);
```

```
            document.getElementById("console").innerHTML =
            '<h2>Все!</h2>';
        } else
            document.getElementById("console").insertAdjacentHTML
            ("beforeend", document.cookie + '<p>');
    }, 2000);
</script>
```

Удаление Cookie

Как было отмечено выше, если времени жизни cookie задан ноль или отрицательное значение, то cookie будет удалено.

example_12. **example_12.html**

```
<a href="set-cookie.html">Установить Cookie</a> |
<a href="del-cookie.html">Удалить Cookie</a>
<script>
    // проверяем cookie сайта на пустоту
    if (document.cookie == '') {
        // вывод в браузер сообщения
        document.write('<h3>Cookie citate не существует или
        успешно удалена! (... а жаль)</h3>');
    } else {
        // вывод в браузер cookie
        // document.write('<h3>',
        decodeURIComponent(document.cookie), '</h3>');
        document.write('<h2>', document.cookie, '</h2>');
    }
```

```
</script>
```

Сценарий установки cookie.

example_12. set-cookie.html

```
<script>

    // сохраняем cookie

    let citate = '"Самое прекрасное в обучении то, что никто не
    может отнять это у тебя". – Б.Б. Кинг';

    // citate = encodeURIComponent(citate);

    document.cookie = `citate=${citate}; path=/`;

    // переадресуем на основную страницу

    window.location.assign(window.location.origin +
    '/example_12.html');

</script>
```

Сценарий удаления cookie.

example_12. del-cookie.html

```
<script>

    // удаляем cookie

    document.cookie = 'citate=; max-age=0';

    // переадресуем на основную страницу

    window.location.assign('/example_12.html');

</script>
```



Cookie и JSON

Сохраняя массивы или объекты можно создавать набор файлов cookie. А можно использовать формат JSON. В демонстрационных примерах **example_13-example_14** попробуем оба подхода.

Создаем набор файлов cookie.

example_13. **example_13.html**

```
<script src="characters.js"></script>
<!-- ... -->
<a href="target.html">Go-go</a><p>
<script>
    // формируем cookie данные в цикле
    for(let key in character) {
        // console.log(key);
        document.cookie = `${key}=${character[key]}; path=/`;
    }
</script>
```

example_13. **target.html**

```
<script>
    // разбиваем cookie по разделителю: ;
    let cookies = document.cookie.split('; ');
    // массив
    console.log(cookies);
    // обходим массив файлов cookie
    for (let cookie of cookies) {
```

```
        let [key, val] = cookie.split('=');

        document.write(key, ' -> ', val, '<p>');
    }
</script>
```

example_13. characters.js

```
let character = {
    id : "5",
    name: "Гань Юй",
    rarity: "5 звезд",
    weapon: "стрелковое",
    element: "крио",
    model: "средняя женская",
    birthday: "5 декабря",
    teazer: "Главный секретарь павильона Лунного моря. В её жилах
    течёт кровь мифического зверя цилиня.",
    content: "Прицельный выстрел Гань Юй можно зарядить дважды.
    При попадании такой стрелы раскроется морозный узор и ...",
    images: "Personazh_Gan_Yuy_Kartochka.jpg"
}
```

Создаем один файл cookie со значением в виде строки формата JSON.

example_14. example_14.html

```
<script src="characters.js"></script>
<!-- ... -->
<a href="target.html">Go-go</a><p>
<script>
```

```
    // сохраняем в виде строки JSON объект character
    let characterJSON = JSON.stringify(character);
    // сохраняем cookie
    document.cookie = `character=${characterJSON}; path=/`;
</script>
```

example_14. target.html

```
<script>
    // извлекаем из cookie строку формата JSON
    let [, val] = document.cookie.split('=');
    // преобразуем JSON в JS-объект
    let character = JSON.parse(val);
    // перебираем свойства объекта
    for (let key in character) {
        // если свойство - изображение
        if(key == "images")
            // вывод изображения
            document.write(`<img src="images/${character[key]}"
            width=300px>`);
        else
            // вывод текстовых данных
            document.write(key, ' -> ', character[key], '<p>')
    }
</script>
```



Сторонние Cookie и безопасность

Директива secure

Синтаксис директивы:

`secure`

Cookie следует передавать только по **HTTPS-протоколу**.

По умолчанию cookie, установленные сайтом **http**://site.com, также будут доступны на сайте **https**://site.com и наоборот. По умолчанию cookie опираются на доменное имя, и не обращают внимания на протоколы.

С настройкой **secure**, если куки будет установлено на сайте https://site.com, то оно не будет доступно на том же сайте с протоколом HTTP.

Таким образом, если в cookie хранится конфиденциальная информация, которую не следует передавать по незашифрованному протоколу HTTP, то нужно установить этот флаг.

```
document.cookie = "user=DenisDream; secure";
```

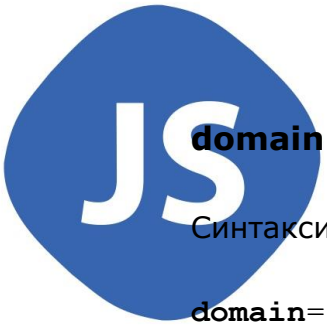
samesite

Синтаксис директивы:

`samesite`

Настройка безопасности, применяется для защиты от так называемой XSRF-атаки (межсайтовая подделка запроса).

> ***Примечание***. Вопросы безопасности настолько обширны, что будут рассматриваться отдельными уроками.



domain

Синтаксис директивы:

domain=site.com

Домен определяет, где доступен файл cookie. На практике существуют определённые ограничения, мы не можем указать какой угодно домен.

Файлы cookie доступны лишь тому домену, который его установил.

Нет никакого способа разрешить доступ к файлам cookie из другого домена, поэтому other.com никогда не получит cookie, установленный на домене site.com.

Это ограничение безопасности, позволяет хранить конфиденциальные данные в файлах cookie, которые должны быть доступны только на одном сайте.

По умолчанию файл cookie **не передаётся поддомену**, например cookie домена site.com не будет передана в поддомен forum.site.com. Если в вашем локальном сервере настроены поддомены, попробуйте эту практику самостоятельно.

Обратите внимание, в следующем демонстрационном примере целевой файл **target.html** я разместил на разных доменах:

```
<a href="/target.html">Текущий домен</a><p>
<a href="http://site/target.html">Сторонний домен</a><p>
```

Тестируем.

example_15. **example_15.html**

```
<h3>Файлы Cookie установлены, перейдите по ссылкам для проверки document.cookie</h3>
<a href="/target.html">Текущий домен</a><p>
<a href="http://site/target.html">Сторонний домен</a><p>
<script>
```

```
    // установка cookie
    document.cookie = `login=DenisDream; path=/; somesite`;
    document.cookie = `pwd=qwerty@; path=/; somesite`;
    document.cookie = `email=denis-dream@gmail.com; path=/;
    somesite; domain=site`;
</script>
```

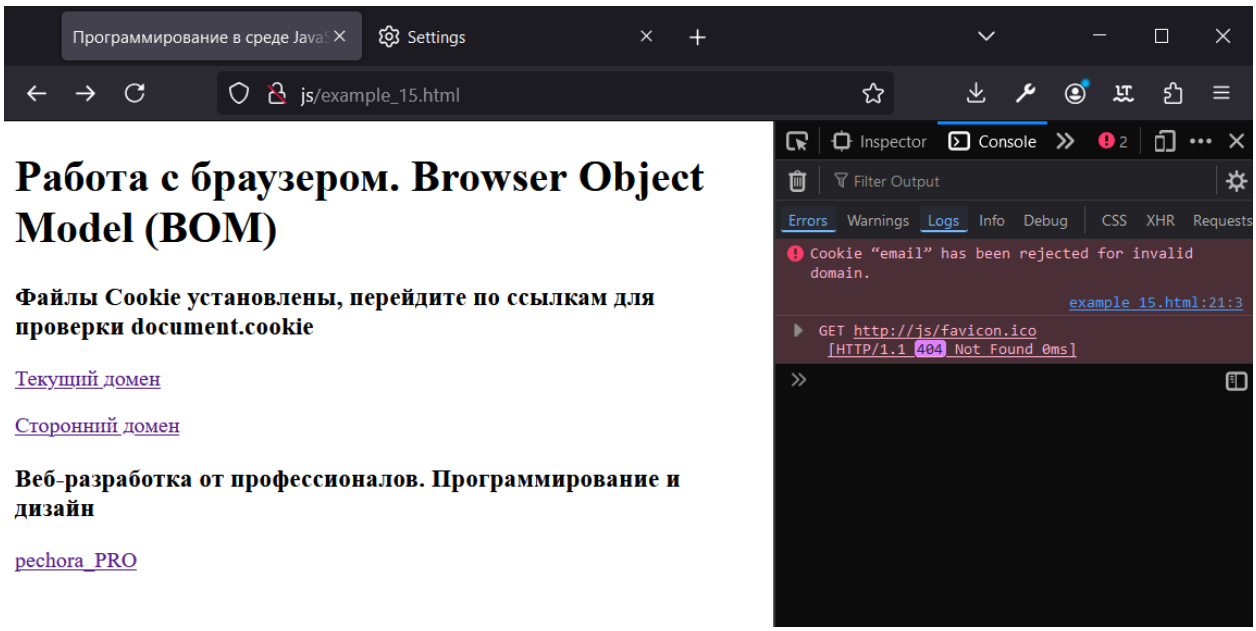
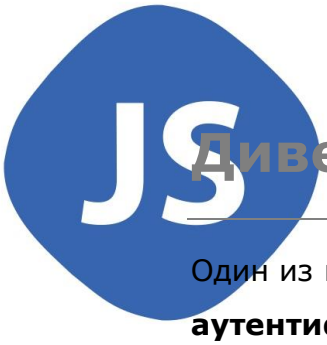


Рис.7. Попытка отправить cookie на сторонний домен

example_15. target.html

```
<script>
    // вывод файлов cookie
    document.write(document.cookie);
    console.log(document.cookie);
</script>
```



Дивертисмент на тему аутентификации

Один из наиболее частых случаев использования cookie – это **аутентификация** пользователя и сбор информации о его действиях на сайте.

При входе пользователя на сайт **сервер** отправляет **HTTP-заголовок Set-Cookie** для того, чтобы установить на компьютере пользователя cookie со специальным уникальным идентификатором сессии.

Во время следующего запроса к этому же сайту **браузер** отправляет на сервер **HTTP-заголовок Cookie**, содержащий ранее установленный идентификатор сессии. Таким образом, сервер понимает, кто сделал запрос.

В демонстрационном примере **example_16** попробуем пофантазировать на тему аутентификации пользователя.

Файлы приложения демонстрационного примера **example_16**:

- **index.html** – стартовая страница сайта;
- **page.html** – общедоступная страница сайта;
- **moderator/index.html** – страница модератора сайта;
- **admin/index.html** – страница администратора сайта;
- **users.js** – массив аутентификационных данных пользователей.



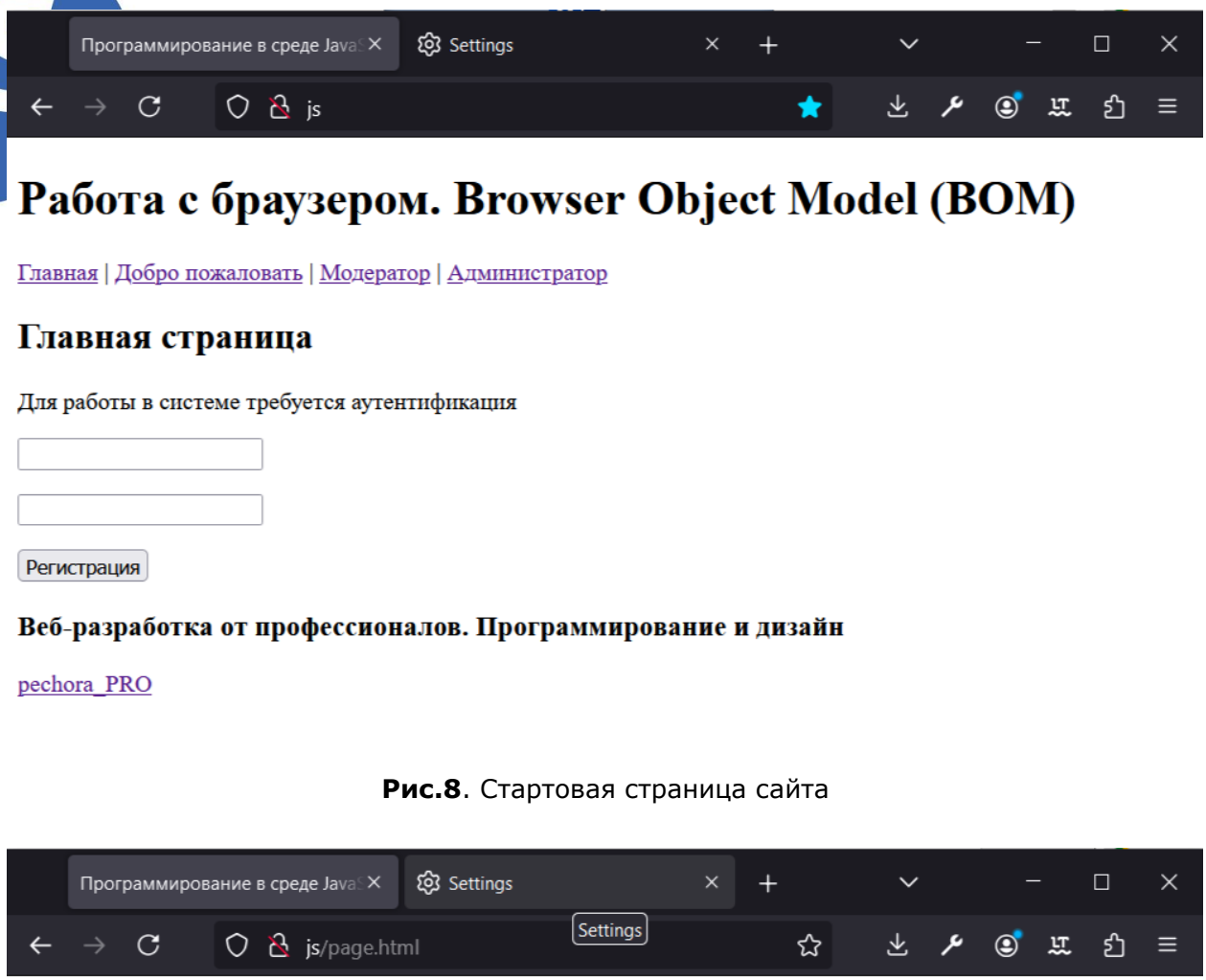


Рис.8. Стартовая страница сайта

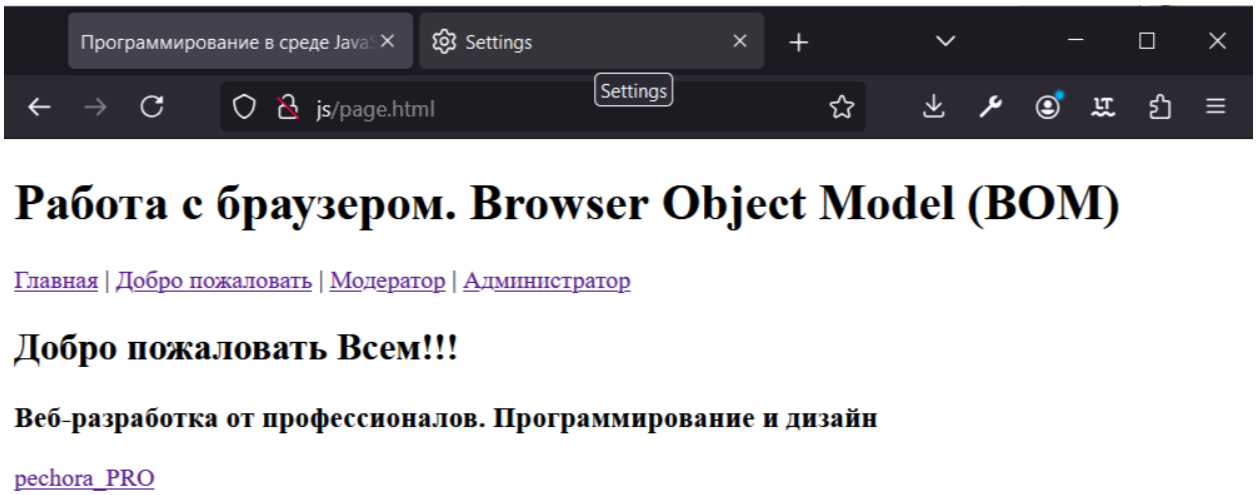


Рис.9. Общедоступная страница сайта

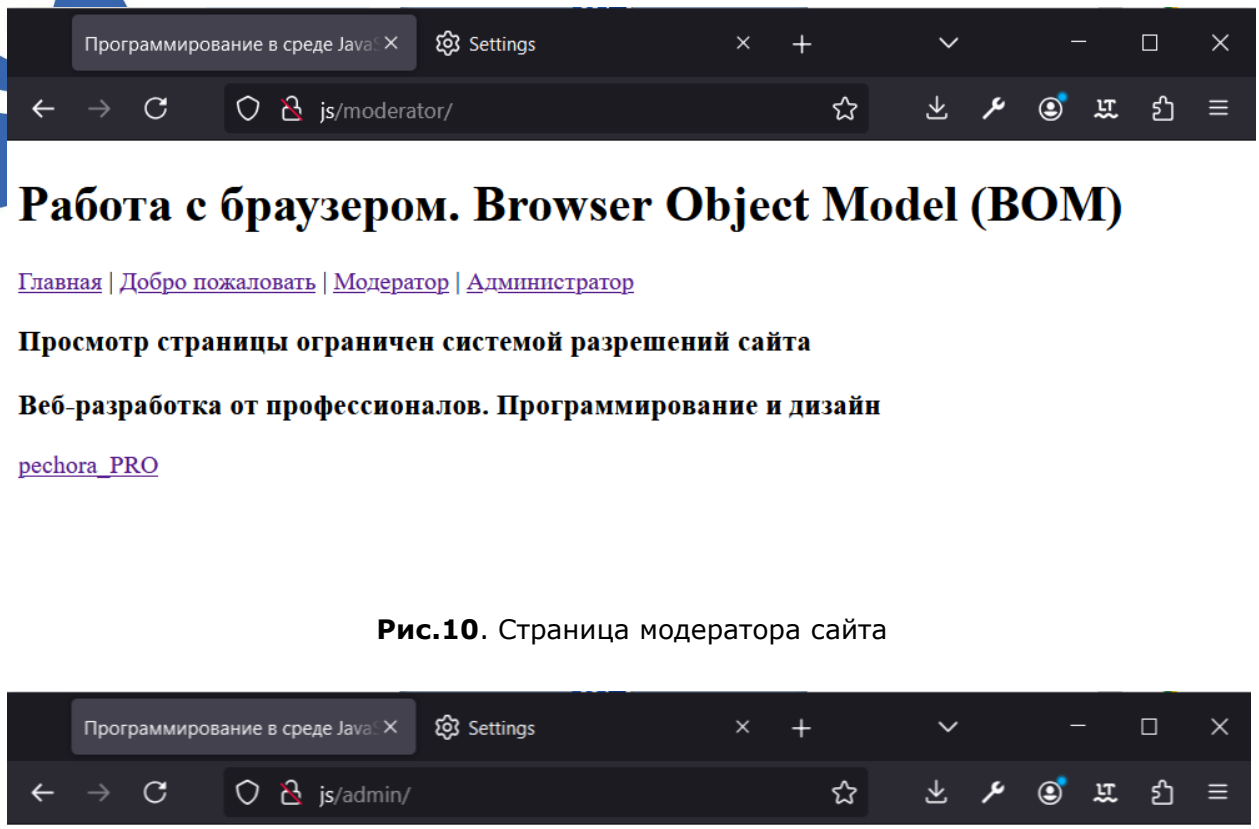


Рис.10. Страница модератора сайта

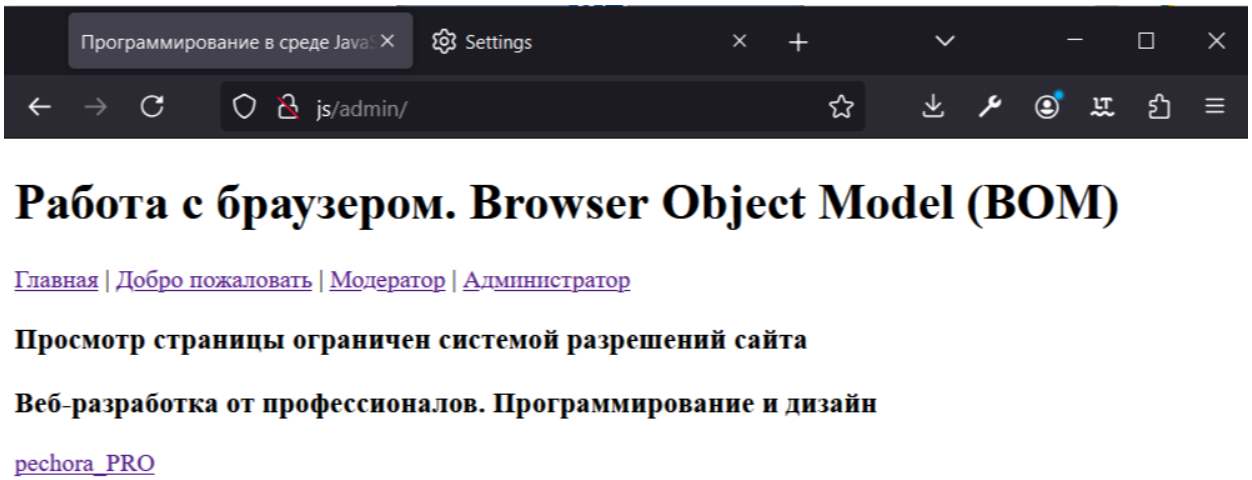


Рис.11. Страница администратора сайта

Обратите внимание, сетевая идентификация пользователя процесс достаточно сложный и затрагивает как клиентские, так и серверные технологии. Поэтому текущий раздел – некоторое развлекательное предположение, как могла бы выглядеть аутентификация без использования серверных технологий, на *чистом* клиенте.

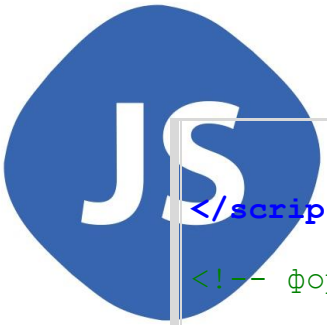
> ***Примечание***. Дивертисмент от фр. divertissement — увеселение, развлечение.



Стартовая страница сайта.

example_16. index.html

```
<script src="users.js"></script>
<!-- ... -->
<a href="/">Главная</a> |
<a href="/page.html">Добро пожаловать</a> |
<a href="/moderator/">Модератор</a> |
<a href="/admin/">Администратор</a><p>
<h2>Главная страница</h2>
<script>
    // инициируем объект, содержащий информацию о текущем
    пользователе
    // (если таковой имеется)
    let user = {};
    // формируем объект с данными из cookie
    let cookies = document.cookie.split('; ');
    for (let cookie of cookies) {
        let [key, val] = cookie.split('=');
        user[key] = val;
    };
    // проверяем роль текущего пользователя
    if (user.role == 'Администратор' || user.role == 'Модератор')
    {
        document.write(`<p>Вы зашли как ${user.role} сайта</p>`);
    } else
        document.write('<p>Для работы в системе требуется
```



```
        аутентификация</p>');
</script>
<!-- форма аутентификации пользователя -->
<form id="frm-auth" onsubmit="return fnAuth()">
    <input type="text" name="login" value=""><p>
    <input type="text" name="pwd" value=""><p>
    <button>Регистрация</button>
</form>
<script>
    // обработчик отправки данных формы
    function fnAuth() {
        let data = new FormData(document.getElementById("frm-
        auth"));
        // если пользователь не ввел данные
        if (
            data.get('login') == '' ||
            data.get('pwd') == ''
        ) {
            alert("Введите данные!");
        } else {
            // если аутентификационные данные введены,
            сопоставляем их
            // с массивом данных пользователей
            for (user of users) {
                // если пользователь в массиве найден
                if (
                    user.login == data.get('login') &&
```

```
                    user.pwd == data.get('pwd')
                ) {
                    alert("Спасибо за регистрацию!");
                    // создаем данные cookie
                    document.cookie = `login=${user.login};
                    path=/`;
                    document.cookie = `pwd=${user.pwd}; path=/`;
                    document.cookie = `role=${user.role};
                    path=/`;
                    // переадресуем на общедоступную страницу
                    window.location.href = '/page.html';
                }
            }
            // если пользователь не найден
            alert("Пользователь с такими данными не найден");
        }
        return false;
    }
</script>
```

Общедоступная страница сайта.

example_16. page.html

```
<a href="/">Главная</a> |
<a href="/page.html">Добро пожаловать</a> |
<a href="/moderator/">Модератор</a> |
<a href="/admin/">Администратор</a><p>
<h2>Добро пожаловать Всем!!!</h2>
```



Страница модератора сайта.

example_16. moderator/index.html

```
<a href="/">Главная</a> |
<a href="/page.html">Добро пожаловать</a> |
<a href="/moderator/">Модератор</a> |
<a href="/admin/">Администратор</a><p>
<script>
    // инициируем объект, содержащий информацию о текущем
    пользователе
    // (если таковой имеется)
    let user = {};
    // формируем объект с данными из cookie
    let cookies = document.cookie.split('; ');
    for (let cookie of cookies) {
        let [key, val] = cookie.split('=');
        user[key] = val;
    };
    // если текущий пользователь - Модератор
    if (user.role == 'Модератор') {
        document.write('<h2>Модератор</h2>');
        document.write('<p>Этот контент для Модератора
        сайта</p>');
        document.write(`<p>Ваш логин: ${user.login}</p>`);
    }
    else {
        document.write('<h3>Просмотр страницы ограничен системой
```

```
        разрешений сайта</h3>');
    }
</script>
```

Страница администратора сайта.

example_16. admin/index.html

```
<a href="/">Главная</a> |
<a href="/page.html">Добро пожаловать</a> |
<a href="/moderator/">Модератор</a> |
<a href="/admin/">Администратор</a><p>
<script>
    // инициируем объект, содержащий информацию о текущем
    пользователе
    // (если таковой имеется)
    let user = {};
    // формируем объект с данными из cookie
    let cookies = document.cookie.split('; ');
    for (let cookie of cookies) {
        let [key, val] = cookie.split('=');
        user[key] = val;
    };
    // если текущий пользователь - Администратор
    if (user.role == 'Администратор') {
        document.write('<h2>Администратор</h2>');
        document.write('<p>Этот контент для Администратора
        сайта</p>');
        document.write(`<p>Ваш логин: ${user.login}</p>`);
```

```
    }
    else {
        document.write('<h3>Просмотр страницы ограничен системой
        разрешений сайта</h3>');
    }
</script>
```

Массив аутентификационных данных пользователей.

example_16. **users.js**

```
let users = [
    {
        login: 'DenisDream',
        pwd: 'qwerty@',
        role: 'Администратор',
        path: 'admin'
    },
    // ...
]
```

Примечание. В самостоятельных работах к уроку продолжим кодинг рассмотренного приложения. Добавим функционал выхода из системы и переадресацию пользователей по результату аутентификации.



Функции для работы с Cookie

Работа с файлами cookie предполагает выполнение некоторого набора стандартных операций. По этой причине опытные разработчики, как правило, сздают набор пользовательских функций для работы с файлами **cookie**.

В этом разделе рассмотрим следующие **пользовательские** функции:

- функция **установки** Cookie;
- функция **вывода** всех Cookie;
- функция **получения** Cookie по ключу;
- функция **проверки** Cookie;
- функция **удаления** Cookie.

При желании пул функций может быть расширен.

Функция установки Cookie (setCookie)

example_17. **example_17.html**

```
<script src="function.js"></script>
<!-- ... -->
<script>
    // установка cookie
    setCookie('login', 'SuperAdmin');
    setCookie('email', 'pechora_pro@gmail.com', 2);
    // вывод cookie
    let cookies = document.cookie.split('; ');
    for (cookie of cookies) {
        [key, val] = cookie.split('=');
        document.write(key, ' -> ', val, '<p>');
    }
```

```
</script>
```

Описание функции установки Cookie – **setCookie**().

example_17. **function.js**

```
<!-- ... -->

// функция установки cookie

function setCookie(name, value, minute=1){

    var d = new Date();

    d.setTime(d.getTime() + (minute * 60 * 1000));

    var expires = "expires=" + d.toUTCString();

    document.cookie = name + "=" + value + "; " + expires + "; 
    path=/; samesite=strict";

};
```

Функция вывода всех Cookie (outCookie)

example_18. **example_18.html**

```
<script src="function.js"></script>

<!-- ... -->

<script>

    // установка cookie

    setCookie('login', 'SuperAdmin');

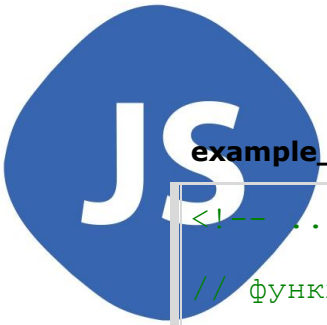
    setCookie('email', 'pechora_pro@gmail.com', 2);

    // вывод cookie

    outCookie();

</script>
```

Описание функции вывода всех Cookie – **outCookie**().



example_18. function.js

```
<!-- ... -->
// функция вывода всех cookie
function outCookie() {
    let cookies = document.cookie.split('; ');
    if(document.cookie != '') {
        for (cookie of cookies) {
            [key, val] = cookie.split('=');
            document.write(key, ' -> ', val, '<p>');
        }
    } else
        document.write("<p>Файлы cookie для страницы не
        установлены</p>");
}
```

Функция получения Cookie по ключу (getCookie)

example_19. example_19.html

```
<script src="function.js"></script>
<!-- ... -->
<script>
    // установка cookie
    setCookie('login', 'SuperAdmin');
    setCookie('email', 'pechora_pro@gmail.com', 2);
    setCookie('pwd', 'root', 1);
    // получение cookie по ключу
    document.write(getCookie('login'), '<p>');
```

```
    document.write(getCookie('email'), '<p>');
    document.write(getCookie('pwd'), '<p>');
</script>
```

Описание функции получения Cookie по ключу – **getCookie**().

example_19. **function.js**

```
<!-- ... -->
// функция получения cookie по ключу
function getCookie(name) {
    let decodedCookie = decodeURIComponent(document.cookie);
    let arr = decodedCookie.split(';');
    /*
        arr[0] = 'login=SuperAdmin'
        arr[1] = ' email=pechora_pro@gmail.com'
        arr[2] = ' pwd=root'
    */
    name = name + "=";
    for(var i = 0; i < arr.length; i++) {
        var cookie = arr[i];
        // избавляемся от возможных ведущих пробелов
        while (cookie.charAt(0) == ' ') {
            cookie = cookie.substring(1);
        }
        // извлекаем значение cookie
        if (cookie.indexOf(name) == 0) {
            return cookie.substring(name.length, cookie.length);
```

```
        }
    }
    return "";
}
```

Функция проверки Cookie (hasCookie)

example_20. **example_20.html**

```
<script src="function.js"></script>
<!-- ... -->
<script>
    // установка cookie
    setCookie('login', 'SuperAdmin');
    setCookie('email', 'pechora_pro@gmail.com', 2);
    setCookie('pwd', 'root', 1);
    // проверка наличия cookie
    hasCookie('email');
    hasCookie('role');
</script>
```

Описание функции проверки Cookie – **hasCookie**().

example_20. **function.js**

```
<!-- ... -->
// функция проверки cookie
function hasCookie(name) {
    var val = getCookie(name);
    if (val != "") {
```

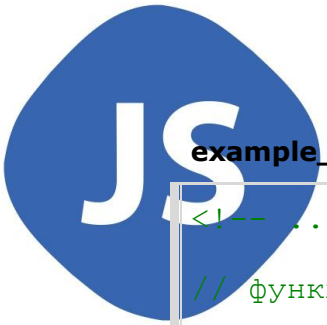
```
        document.write("<p>Cookie " + name + " = " + val);
    } else {
        document.write("<p>Cookie с именем " + name + " на
        странице не найден");
    }
}
```

Функция удаления Cookie (delCookie)

example_21. **example_21.html**

```
<script src="function.js"></script>
<!-- ... -->
<script>
    // установка cookie
    setCookie('login', 'SuperAdmin');
    setCookie('email', 'pechora_pro@gmail.com', 2);
    setCookie('pwd', 'root', 1);
    // проверка cookie
    hasCookie('email');
    // удаление cookie
    delCookie('email');
    // проверка cookie
    hasCookie('email');
</script>
```

Описание функции удаления Cookie – **delCookie**().



example_21. function.js

```
<!-- ... -->

// функция удаления cookie

function delCookie(name) {

    setCookie(name, "", -1);

}
```

Больше примеров и практики в **самостоятельных работах**:

- Создание файлов Cookie и преадресация в целевой файл в зависимости от состояния таймера.
- Создание функционала выхода пользователя из системы.
- Управление видимостью кнопки выхода.
- Переадресация пользователей в зависимости от результата аутентификации.

Примечание. Программный код самостоятельных работ выполнен с целью обеспечения наилучшего визуального восприятия алгоритма приложения. Дальнейшее ревьюирование и *накрутка* сложности кода приветствуются.

P.S.

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).





К каждой **теме** курса:

- **Практические работы**.

К **курсу**:

- **Дополнительный практикум.**

